

MASTER PLAN V2 – Radar tầm gần STM32F429I-DISC1 bản hoàn thiện

1. Mục tiêu chính

Tên đề tài: Radar tầm gần

Board:

- STM32F429I-DISC1
- MCU STM32F429ZIT6
- LCD TFT 2.4 inch QVGA 320x240 có sẵn trên kit
- TouchGFX Designer dùng để thiết kế giao diện
- STM32CubeMX + STM32CubeIDE tách riêng

Mục tiêu demo:

- Cảm biến SR-04 / HC-SR04 đo khoảng cách vật cản.
- Servo MG90S quay cảm biến theo góc quét 90° hoặc 180°.
- LCD TouchGFX hiển thị radar đẹp, có tia quét, góc, khoảng cách, vật cản.
- OLED SH1106 hiển thị phụ thông tin góc/khoảng cách/trạng thái.
- Buzzer active kêu khi vật cản quá gần.
- LED3/LED4 trên board báo trạng thái.
- B1 dùng đổi tốc độ quét hoặc chuyển lựa chọn trong cài đặt.
- Có màn hình Home, Scan, Warning, Settings, Info.
- Code chia module rõ ràng để dễ viết báo cáo và đưa lên GitHub.

2. Linh kiện dùng chính thức

2.1 Bắt buộc

1. STM32F429I-DISC1
2. Board xử lý trung tâm.
3. Có LCD TouchGFX.
4. Có LED3, LED4, nút B1.
5. SR-04 / HC-SR04
6. Cảm biến siêu âm đo khoảng cách.
7. Dùng Trigger + Echo.

8. Servo MG90S
9. Servo quay cảm biến radar.
10. Điều khiển bằng PWM 50 Hz.
11. OLED SH1106 0.96 inch I2C
12. Màn phụ hiển thị trạng thái nhanh.
13. Hiển thị angle, distance, mode, warning.
14. Buzzer active
15. Cảnh báo khi vật cản gần.
16. Chỉ cần GPIO mức 1/0 để bật/tắt.

2.2 Linh kiện phụ cần có

- Điện trở chia áp Echo SR-04 xuống 3.3V.
- Dây Dupont.
- Nguồn 5V đủ dòng cho servo.
- GND chung giữa servo, SR-04, OLED, buzzer và STM32.
- Có thể thêm tụ 100uF–470uF gần nguồn servo để giảm sụt áp.

3. Cảnh báo phần cứng

3.1 SR-04 / HC-SR04

- VCC: 5V.
- GND: GND chung.
- TRIG: STM32 output 3.3V.
- ECHO: tín hiệu trả về 5V, không cắm thẳng vào STM32.
- Phải chia áp Echo xuống khoảng 3.3V.

Ví dụ chia áp:

- Echo SR-04 -> R1 1k -> điểm giữa -> chân Echo STM32.
- Điểm giữa -> R2 2k -> GND.
- Điện áp vào STM32 khoảng 3.3V.

3.2 MG90S servo

- Servo nên cấp 5V riêng hoặc từ nguồn 5V đủ dòng.
- Không cấp servo từ chân GPIO.
- GND servo phải nối chung với GND STM32.

- Signal servo nối vào chân PWM của STM32.
- PWM servo:
 - Tần số: 50 Hz.
 - Chu kỳ: 20 ms.
 - 0° khoảng 0.5 ms.
 - 90° khoảng 1.5 ms.
 - 180° khoảng 2.5 ms.
- Có thể cần hiệu chỉnh MIN/MAX pulse tùy servo thực tế.

3.3 OLED SH1106

- VCC: 3.3V.
- GND: GND.
- SCL/SDA nối I2C.
- Nên dùng I2C3 nếu không xung đột với thiết kế hiện tại.

3.4 Buzzer active

- Nếu buzzer active 3.3V: có thể điều khiển trực tiếp qua GPIO nếu dòng nhỏ.
- Nếu buzzer 5V hoặc dòng lớn: nên dùng transistor.
- Buzzer active chỉ cần bật/tắt, không cần PWM âm thanh.

4. Hướng triển khai tổng thể

Ưu tiên làm theo thứ tự:

1. Thiết kế giao diện TouchGFX trước.
2. Dùng GPT/image_gen tạo asset ảnh đẹp cho giao diện.
3. Import asset vào TouchGFX.
4. Tạo UI skeleton đầy đủ màn hình.
5. Tạo cầu nối dữ liệu UI.
6. Viết driver servo MG90S.
7. Viết driver SR-04.
8. Viết driver OLED SH1106.
9. Viết driver buzzer/LED.
10. Viết logic radar app.
11. Ghép FreeRTOS task.
12. Ghép dữ liệu sang TouchGFX.
13. Build full.
14. Test từng module nếu phần cứng lỗi.
15. Hoàn thiện báo cáo GitHub.

5. Vì sao làm TouchGFX trước?

Đề tài này cần demo nhìn đẹp, nên UI quyết định ấn tượng đầu tiên.

Làm TouchGFX trước có lợi:

- Biết trước cần dữ liệu gì từ code.
- Biết tên object UI để code bấm theo.
- Tránh viết logic xong rồi không biết đổ dữ liệu ra màn hình thế nào.
- Dễ tạo mockup đẹp bằng GPT/image_gen.
- Dễ chia module: code chỉ cần cung cấp angle, distance, status, mode.

Dữ liệu UI cần từ code:

- current_angle
- current_distance
- scan_mode
- speed_mode
- object_detected
- near_warning
- object_count
- min_distance
- buzzer_state
- led_state
- oled_state

6. Asset ảnh cần tạo bằng GPT/image_gen

6.1 Asset nền

Cần tạo:

- background_home.png
- background_scan.png
- background_warning.png
- background_settings.png
- background_info.png

Kích thước:

- 320x240 px hoặc lớn hơn rồi resize về 320x240.

Phong cách:

- Embedded radar UI.
- Tông xanh đậm / đen / neon green.
- Warning dùng đỏ/vàng.
- Không giống app điện thoại.
- Phù hợp LCD 2.4 inch.

6.2 Asset icon

Cần tạo:

- icon_radar.png
- icon_play.png
- icon_settings.png
- icon_info.png
- icon_warning.png
- icon_buzzer.png
- icon_servo.png
- icon_distance.png
- icon_oled.png
- icon_led.png

Yêu cầu:

- Icon đơn giản, dễ nhìn ở kích thước nhỏ.
- Nền trong suốt nếu có thể.
- Đồng bộ phong cách.

6.3 Asset radar

Có thể tạo:

- radar_grid_180.png
- radar_grid_90.png
- target_dot.png
- sweep_glow.png

Lưu ý:

- Tia quét và target có thể vẽ bằng code TouchGFX để linh hoạt.
- Nền radar grid có thể dùng ảnh để đẹp hơn.

7. Giao diện TouchGFX cần có

7.1 ScreenHome

Tên màn hình:

- ScreenHome

Object gợi ý:

- imgHomeBg
- imgRadarLogo
- btnStart

- btnSettings
- btnInfo
- txtTitle
- txtSubtitle

Chức năng:

- btnStart -> ScreenScan
- btnSettings -> ScreenSettings
- btnInfo -> ScreenInfo

7.2 ScreenScan

Tên màn hình:

- ScreenScan

Object gợi ý:

- imgScanBg
- imgRadarGrid
- lineSweep hoặc custom container vẽ tia
- imgTarget
- txtAngle
- txtDistance
- txtStatus
- txtMode
- btnBackHome
- btnSettingsSmall

Chức năng:

- Hiển thị radar đang quét.
- Nếu phát hiện vật cản thì hiện target.
- Nếu vật cản gần thì chuyển hoặc báo ScreenWarning.

7.3 ScreenWarning

Tên màn hình:

- ScreenWarning

Object gợi ý:

- imgWarningBg
- imgWarningIcon
- imgBuzzerIcon
- txtWarningTitle
- txtWarningDistance

- txtWarningAngle
- btnBackScan

Chức năng:

- Hiển thị cảnh báo vật cản gần.
- Buzzer bật.
- LED4 bật hoặc nháy.

7.4 ScreenSettings

Tên màn hình:

- ScreenSettings

Object gợi ý:

- imgSettingsBg
- btnSpeedSlow
- btnSpeedMedium
- btnSpeedFast
- btnMode90
- btnMode180
- txtHintB1
- btnBackHome

Chức năng:

- Chọn tốc độ servo.
- Chọn góc quét 90° / 180°.
- B1 có thể chuyển lựa chọn.
- Touch cũng có thể chọn nếu dùng cảm ứng.

7.5 ScreenInfo

Tên màn hình:

- ScreenInfo

Object gợi ý:

- imgInfoBg
- txtScanMode
- txtSpeedMode
- txtMinDistance
- txtLastAngle
- txtObjectCount
- txtBuzzerState
- txtLedState

- txtOledState
- btnBackHome

Chức năng:

- Hiển thị thông kê hệ thống.
- Dùng cho báo cáo/demo.

8. Kiến trúc code module

8.1 File cấu hình

Core/Inc/radar_config.h

Chứa:

- Ngưỡng phát hiện vật cản.
- Ngưỡng cảnh báo gần.
- Góc quét.
- Tốc độ quét.
- Pulse min/max servo.
- Bật/tắt OLED.
- Bật/tắt UART debug.

8.2 Driver servo

Core/Inc/servo_mg90s.h

Core/Src/servo_mg90s.c

Hàm chính:

- Servo_Init()
- Servo_SetAngle(uint16_t angle)
- Servo_SetPulseUs(uint16_t pulse_us)

Kiến trúc:

- Timer PWM.
- CCR thay đổi độ rộng xung.
- Servo đọc độ rộng xung để xoay góc.

8.3 Driver SR-04

Core/Inc/hcsr04.h

Core/Src/hcsr04.c

Hàm chính:

- HCSR04_Init()
- HCSR04_ReadDistanceCm(uint16_t *distance)
- HCSR04_ReadRawUs(uint32_t *echo_us)

Kiến thức:

- Trigger 10 us.
- Đo Echo pulse width.
- $\text{distance_cm} = \text{echo_us} / 58$.
- Timeout tránh treo.

8.4 Driver buzzer/LED

Core/Inc/buzzer_led.h

Core/Src/buzzer_led.c

Hàm chính:

- BuzzerLed_Init()
- Buzzer_Set(uint8_t on)
- LedScan_Set(uint8_t on)
- LedDetect_Set(uint8_t on)
- Alert_Update(uint8_t detected, uint8_t near)

8.5 Driver OLED SH1106

Core/Inc/oled_status.h

Core/Src/oled_status.c

Dùng thư viện:

- SH1106.c
- SH1106.h
- fonts.c
- fonts.h

Hàm chính:

- OledStatus_Init()
- OledStatus_ShowNormal(angle, distance, mode)
- OledStatus_ShowDetected(angle, distance)
- OledStatus_ShowWarning(angle, distance)
- OledStatus_ShowError()

8.6 UI bridge

Core/Inc/radar_ui_bridge.h

Core/Src/radar_ui_bridge.c

Struct:

```
typedef struct { uint16_t angle_deg; uint16_t distance_cm; uint8_t object_detected; uint8_t near_warning;
uint8_t scan_mode_deg; uint8_t speed_mode; uint16_t object_count; uint16_t min_distance_cm; uint16_t
last_detect_angle; uint8_t buzzer_on; uint8_t led3_on; uint8_t led4_on; uint8_t oled_connected; }
RadarUiData_t;
```

Hàm:

- RadarUiBridge_SetData(const RadarUiData_t *data)
- RadarUiBridge_GetData(RadarUiData_t *data)

Nhiệm vụ:

- Cho C code cập nhật dữ liệu.
- Cho TouchGFX C++ đọc dữ liệu.

8.7 Radar app

Core/Inc/radar_app.h

Core/Src/radar_app.c

Hàm chính:

- RadarApp_Init()
- RadarApp_TaskLoop()
- RadarApp_SetSpeedMode()
- RadarApp_SetScanMode()
- RadarApp_GetState()

Logic:

- Quét từ trái sang phải.
- Đến biên thì đảo chiều.
- Mỗi bước:
 - Set servo angle.
 - Chờ servo ổn định.
 - Đọc khoảng cách.
 - Lọc dữ liệu.
 - Kiểm tra vật cản.
 - Cập nhật LED/buzzer.
 - Cập nhật OLED.
 - Cập nhật UI bridge.

9. FreeRTOS task

9.1 RadarTask

Nhiệm vụ:

- Chạy logic radar chính.
- Gọi RadarApp_TaskLoop().
- Chu kỳ tùy speed mode.

9.2 ButtonTask

Nhiệm vụ:

- Đọc B1.
- Chống dội.
- Đổi tốc độ hoặc mode tùy màn hình.
- Có thể gửi command vào queue.

9.3 TouchGFX task

Nhiệm vụ:

- TouchGFX xử lý giao diện.
- Model đọc RadarUiBridge_GetData().
- View cập nhật text/icon/target.

9.4 UartDebugTask optional

Nhiệm vụ:

- In log:
- angle
- distance
- detected
- warning
- speed
- mode

10. Thứ tự triển khai chi tiết

Phase 0: Chốt project nền

Quyết định:

- Dùng project TouchGFX đã chạy sạch làm nền.
- Copy thành project radar mới.

- Không giữ logic game cũ.
- Chỉ giữ cấu hình LCD, TouchGFX, FreeRTOS, clock, LTDC, SDRAM.

Phase 1: Thiết kế asset bằng GPT/image_gen

Tạo:

- Mockup tổng thể.
- Background Home.
- Background Scan.
- Background Warning.
- Background Settings.
- Background Info.
- Icon radar, warning, buzzer, servo, OLED.

Xuất:

- PNG 320x240 cho background.
- PNG icon nền trong suốt nếu có thể.

Phase 2: Tạo UI TouchGFX

Trong TouchGFX Designer:

- Tạo ScreenHome.
- Tạo ScreenScan.
- Tạo ScreenWarning.
- Tạo ScreenSettings.
- Tạo ScreenInfo.
- Import image assets.
- Đặt object đúng tên.
- Tạo navigation:
- Home -> Scan
- Home -> Settings
- Home -> Info
- Settings -> Home
- Info -> Home
- Warning -> Scan/Home
- Generate code.
- Build sạch.

Phase 3: Tạo UI bridge

Tạo:

- radar_ui_bridge.h
- radar_ui_bridge.c

Mục tiêu:

- TouchGFX có thể đọc dữ liệu giả.
- Ban đầu chưa cần phần cứng thật.
- Có thể simulate angle/distance để test UI.

Phase 4: Code TouchGFX đọc dữ liệu

Sửa:

- Model.hpp / Model.cpp
- ScreenScanPresenter
- ScreenScanView
- ScreenInfoView
- ScreenWarningView
- ScreenSettingsView

Mục tiêu:

- ScreenScan hiện angle/distance/status.
- Target hiện/ẩn theo object_detected.
- Warning hiện khi near_warning.
- Info hiện thống kê.

Phase 5: Cấu hình CubeMX phần cứng

Cần cấu hình:

- Servo MG90S: 1 chân Timer PWM.
- SR-04 Trigger: GPIO Output.
- SR-04 Echo: GPIO Input hoặc Timer Input Capture.
- Buzzer active: GPIO Output.
- LED3 PG13: GPIO Output.
- LED4 PG14: GPIO Output.
- B1 PA0: GPIO Input.
- I2C cho OLED SH1106.
- UART debug nếu cần.

Ưu tiên:

- Không đổi cấu hình TouchGFX/LCD đang chạy.
- Chỉ thêm peripheral cần thiết.

Phase 6: Viết driver servo MG90S

Tạo:

- servo_mg90s.c/h

Test bằng code:

- 0°
- 90°
- 180°
- sweep 0 -> 180 -> 0

Nếu cần project test riêng:

- radar_test_servo_mg90s

Phase 7: Viết driver SR-04

Tạo:

- hcsr04.c/h

Test:

- Đo khoảng cách vật trước cảm biến.
- In UART.
- Hiển thị OLED/LCD nếu muốn.

Nếu cần project test riêng:

- radar_test_hcsr04

Phase 8: Viết driver OLED SH1106

Tạo:

- oled_status.c/h
- Thêm SH1106.c/h và fonts.c/h nếu chưa có.

Test:

- OLED hiện:
- RADAR
- Angle
- Distance
- Status

Nếu cần project test riêng:

- radar_test_oled_buzzer

Phase 9: Viết driver buzzer/LED

Tạo:

- buzzer_led.c/h

Test:

- LED3 bật khi scanning.
- LED4 bật khi detected.
- Buzzer bật khi warning.

Phase 10: Viết radar_app

Tạo:

- radar_app.c/h

Logic:

- Quét theo scan_mode.
- Tăng/giảm angle.
- Điều khiển servo.
- Đọc SR-04.
- Detect object.
- Warning nếu gần.
- Update OLED.
- Update LED/buzzer.
- Update UI bridge.

Phase 11: Ghép FreeRTOS

Trong main.c hoặc freertos.c:

- Tạo RadarTask.
- Tạo ButtonTask.
- Giữ TouchGFX task.
- RadarTask gọi RadarApp_TaskLoop().
- ButtonTask đọc B1 và đổi speed/mode.

Phase 12: Build full

Mục tiêu:

- Build sạch.
- Không lỗi include.
- Không lỗi C/C++ linkage.
- Không lỗi TouchGFX object name.

- Không lỗi duplicate function.

Phase 13: Test trên board

Thứ tự test: 1. LCD TouchGFX lên Home. 2. Chuyển màn hình Scan. 3. Servo quay. 4. SR-04 đọc khoảng cách. 5. OLED hiện dữ liệu. 6. LED3 báo đang quét. 7. LED4 báo phát hiện. 8. Buzzer kêu khi vật gần. 9. Info screen cập nhật thống kê. 10. Settings đổi speed/mode.

Phase 14: Chuẩn bị báo cáo

Theo format GitHub ProjectSample:

- Giới thiệu đề tài.
- Tác giả.
- Môi trường hoạt động.
- Linh kiện sử dụng.
- Sơ đồ kết nối.
- Nguyên lý hoạt động.
- Thiết kế phần mềm.
- Đặc tả hàm.
- Kết quả đạt được.
- Ảnh/video demo.
- Hướng phát triển.

11. Code full trước hay test trước?

Chiến lược chọn:

- Tạo code full theo module trước.
- Build sạch project chính trước.
- Nếu phần cứng lỗi thì tách project test nhỏ để debug từng module.

Lý do:

- Bạn cần có bản hoàn thiện sớm.
- Code module hóa giúp tách test dễ.
- Project test chỉ là công cụ phụ, không phải hướng chính.

12. Thứ tự thực hiện ngay bây giờ

Bước 1:

- Chốt project nền.
- Copy project TouchGFX đã build sạch sang project radar mới.
- Build lại project radar mới.

Bước 2:

- Tạo asset giao diện bằng GPT/image_gen.

Bước 3:

- Import asset vào TouchGFX.
- Tạo các screen đúng tên.

Bước 4:

- Generate code.
- Build sạch.

Bước 5:

- Tạo radar_ui_bridge.
- Cho UI chạy dữ liệu giả.

Bước 6:

- Thêm driver servo, SR-04, OLED, buzzer.

Bước 7:

- Ghép radar_app.

Bước 8:

- Ghép FreeRTOS task.

Bước 9:

- Test board thật.

13. Quy tắc hỗ trợ tiếp theo

- Mỗi bước hướng dẫn khoảng 5–6 thao tác.
- Sau mỗi bước dừng lại chờ người dùng nhấn OK.
- Khi sửa code phải nói rõ:
 - file nào
 - đoạn nào
 - chèn/thay thế gì
- Không nhảy bước.
- Không tự đổi board.
- Không tự đổi pin nếu chưa báo.
- Code tự viết ưu tiên file .c/.h riêng hoặc USER CODE.
- Nếu có lỗi build, sửa theo lỗi thực tế.
- Vừa code vừa giải thích kiến thức vừa đủ để học.